

2024 年南京邮电大学校大学生科学技术协会 软件研发部招新试题（开卷）

2024 年 9 月

注意事项

1. 欢迎报名校科协软件研发部！本卷考试形式为**开卷**，鼓励使用搜索引擎来搜索学习。比起试卷的答案，我们更看重的是你学习的过程。但是请**独立完成本卷**，请勿以任何形式抄袭他人的答案。
2. 本卷分为通识和提高两部分。通识部分考察计算机基础知识和搜索能力，提高部分考察专业知识和学习能力。我们鼓励大家在能力范围内完成**尽可能多的题目**。
3. 本卷所有题目的答案应当填写到对应答题卡上，提交时只认可答题卡上的答案。
4. 本卷仅是一张普通的笔试试卷，我们希望通过这张试卷对你有一个初步的了解和认识。所以在答题的时候无需有压力，尽力而为即可。**不管答题情况如何，都请自信地交卷。**

0. 请列举你曾经的编程经历或者相关作品，我们将酌情给分。

通识部分

1. (10 分) 尝试用给定的任意一门编程语言实现在屏幕上输出 "Hello SAST"。(10 分)
可用的语言：C、C++、Java、C#、Rust、JavaScript、Python
2. (10 分) Alice 想要开设个人博客网站，于是开始学习网络相关知识，并在某论坛查找相应资源。
 - (1) Alice 找到了一个资源链接，但是服务器找不到请求的资源，此时服务器会返回什么状态码？(2 分)
 - (2) Alice 关掉电脑，睡了一觉，第二天再次打开论坛的时候发现网站记住了她的登录状态，没有要求重新登录，保留登录状态的功能通常使用了什么？(2 分)
 - (3) Alice 实在找不到她想要的资源，于是在论坛上发帖询问，此时建议用什么请求方式向服务器发送帖文以创建新帖子？(3 分)
 - (4) Alice 发帖的时候，浏览器会先通过域名获取论坛服务器的 IP 地址，然后再将数据发送到相应的 IP 地址。请问如何从命令行模拟获取 IP 的过程？请以查询域名 `www.njupt.edu.cn` 的 IP 地址为例。(3 分)

3. (10 分) Bob 在他的电脑上翻出了很久之前下载的一个小游戏, 学过编程的 Bob 想知这款游戏是如何编写的, 但是目前只有编译后的游戏文件, 并没有源代码。幸运的是, 他在网络上了解到“逆向工程”可以解决这一问题。他还搜索到该领域的一些常用工具:

(A) IDA Pro (B) OllyDbg (C) x64dbg (D) apktool (E) jadx (F) WinHex

已知游戏运行于 64 位 Windows 系统。(填入序号即可)

(1) Bob 希望反编译游戏文件, 即从二进制机器码还原为高级语言代码。他应该从上面的工具中选择 _____ (单选) (2 分)

(2) 读完代码, Bob 想通过直接调试游戏文件来深入了解其中一段代码的功能。他需要选择的动态调试工具是 _____ (单选) (3 分)

(3) Alice 编写了一个简单的安卓程序。Bob 听说后, 想要尝试逆向一下。可以帮助他逆向分析安卓 apk 文件的工具是 _____ (多选) (5 分)

4. (10 分) 对于现代显示器, 屏幕通常由像素组成, 而一个像素通常由三个更小的发光组件: 红色、绿色、蓝色组成, 即光学三原色, 三原色两两组合可以获得洋红, 青, 黄三种间色。我们用一个长度为 3 的二进制码表示颜色, 其中每一位与光学三原色按顺序一一对应, 如洋红可表示为 101。我们可以对颜色进行位运算获得其他颜色, 题目不对位移运算的偏移量作限制。

(1) 请求出 $\sim 000 \wedge 001$ 的色值。(1 分)

(2) 请用两个白色编码值、三个位运算列出一个表达式, 其结果为洋红色编码值。(3 分)

(3) 请用一个洋红色编码值、一个白色编码值、两个位运算列出一个表达式, 其结果为青色编码值。(3 分)

(4) 请用一个洋红色编码值、一个蓝色编码值、两个位运算列出一个表达式, 其结果为青色编码值。(3 分)

5. (10 分) 阮小妍初学 C#, 对顶级语句 (Top-level Statements) 一见钟情。她对这个 C# 9.0 之后出现的新特性非常感兴趣。在知道下面的代码:

```
Console.WriteLine("Hello, SAST!");
```

能够直接输出 Hello, SAST! 之后, 她迫不及待地想知道:

```
Console.WriteLine(args);
```

会输出什么, 你能告诉她吗? (3 分) 为什么? (7 分)

6. (10 分) 正则表达式 (regex), 用于描述文本模式 (pattern) 的语言, 是匹配和操作文本的强大工具, 可以用于查找文本、提取信息、替换内容、验证文本格式。

在 BiliBili 中, 我们可以使用正则表达式来过滤低质量的弹幕, 现在想要屏蔽任意部分连续重复 3 次及以上的弹幕, 例如“甜豆腐脑第一! 甜豆腐脑第一! 甜豆腐脑第一!”, 请问应当怎么写? (10 分)

7. (10 分) 有个小偷想去珠宝店盗窃一番, 它的袋子容量为 10, 现在知道有 4 样东西可以偷 (各只有一件), 它们的重量和价值分别为 (2,1), (3,3), (4,5), (7,9)。((x,y) 表示物品的重量, y 表示价值) 请问在不超过袋子重量的情况下, 小偷能偷的最大价值为多少? 请写出你的思路。(10 分)

8. (10 分) 阮小妍最近在研究光线追踪技术, 现在她需要写出一个方法, 来计算一束光线与一个球体相交的情况。我们假设一个球体需要两个参数来约束: 半径与球心, 我们记为 r 与 c , 其中, r 为一个实数, c 为一个三维点)

(1) 阮小妍先研究了如何表示一个点是否在球面上, 她注意到: 对于任意一点 P , 该点距离球心的距离恰好等于 r 的时候, 即说明该点在球面上。根据点积的知识, 写出球面方程。(2 分)

(2) 为了表达直线相交的情况, 用 $P(t) = O + td$ 替换上一小题方程中的 P , 其中 O 表示直线的起点, d 表示直线的方向。我们将 t 视为主元, 将替换后的方程展开成一元二次方程 $ax^2 + bx + c = 0$ 的形式, 并代入以下条件 $O(1, 1, 4), d(5, 1, 4), c(1, 9, 1), r = 9$, 求直线与球体的交点数量。(4 分)

(3) 简单论述如何判断光线是否与三角形相交? (不用给出计算过程) (4 分)

9. (10 分) 阮小妍最近迷上了前端。通过学习, 她了解到, HTML、CSS 和 JavaScript 这传说中的前端三件套, 可以打造出炫酷的网页, 同时她也看到了校科协学长的优秀作品。于是她准备模仿其中一个作品, 创建一个登录页面, 请你和她一起解决在开发过程中遇到的问题。阮小妍从最基础的内容入手, 编写了一个简单的登录表单, 但还缺少一个提交按钮, 她知道普通的按钮怎么写, 但是在表单中的按钮应该是什么样的呢。

(1) 请在表单中添加一个提交按钮, 使得表单可以提交用户输入的数据。(3 分)

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <title>登录页面</title>
</head>
<body>
  <form action="/login" method="POST">
    <input type="text" placeholder="用户名"
      name="username"><br>
    <input type="password" placeholder="密码"
      name="password"><br>
    <!-- 在这里添加一个提交按钮 -->

  </form>
</body>
</html>
```

在添加按钮后, 阮小妍觉得整个页面不够美观, 为了让页面的布局更加合理, 她希望登录表单能够在页面中居中显示。

(2) 请为表单添加 CSS 样式，使其水平居中。(3 分)

```
<style>
  form {
    width: 300px;
    /* 在这里添加样式使表单水平居中 */

  }
  input {
    width: 100%;
    margin-bottom: 10px;
  }
</style>
```

有一位学长告诉她，在实际开发中，用户输入的有效性验证非常重要，通常会在前端先进行一次检验。所以阮小妍需要一个 JavaScript 函数来获取表单中的用户名和密码输入框，进而获取输入的值，在验证输入是否符合要求后使用 `alert` 向用户反馈，如果验证通过，显示“登录成功”的提示，否则显示“用户名或密码错误”的提示并阻止表单提交。经过查询，阮小妍得知可以使用 `querySelector` 选择用户名和密码的输入框，并写好了一个输入检验函数 `validateCredentials`。

(3) 请将代码片段补全。(4 分)

```
<script>
  // 提供的验证函数，假设用户名和密码都为 'admin'
  function validateCredentials(username, password) {
    return username === 'admin' && password === 'admin';
  }
  document.addEventListener('DOMContentLoaded', function() {
    // 在这里使用 querySelector 选择用户名和密码的输入框

    document.querySelector('form').onsubmit =
      function(event) {
        const username = usernameInput.value;
        const password = passwordInput.value;
        // 在这里调用验证函数

      }
  });
</script>
```

10. (10 分) 阮小妍最近沉迷学习 C 语言, 她学习了函数的知识, 知道了头文件与源文件的概念, 并明白了声明与定义分离的思想。

如下是一个用于判断闰年的函数, 当输入年份为闰年时返回 1, 否则返回 0。函数的声明部分位于 `tools.h` 中, 定义位于 `tools.c` 中, 两者都位于同一目录下。

`tools.h`

```
int isLeapYear(int year);
```

`tools.c`

```
#include "tools.h"

int isLeapYear(int year) {
    if (year % 4 == 0) {
        if (_____) {
            _____; // 是闰年
        }
    }
    return 0; // 不是闰年
}
```

(1) 阮小妍不知道判断闰年的具体条件, 请你帮她完善函数的定义部分。(2 分)

现在, 阮小妍想要使用这个函数来完成一个名为 `program` 的程序, 在上述两个文件同一目录下编写了 `program.c`

```
#include <stdio.h>

int main() {
    int year;
    scanf("%d", &year);
    if (isLeapYear(year)) {
        printf("Yes!\n");
    }
    return 0;
}
```

(2) 阮小妍使用 `gcc` 作为编译器, 请你查阅 `gcc` 的使用手册和 C 语言的编译流程, 告诉她如何使用一条命令就能将上面的代码编译成可执行文件 `program`。(2 分)

(3) 阮小妍经过尝试, 发现源代码有误导致编译失败, 请指出错误 (3 分)

阮小妍发现构建上述程序有点麻烦, 每次得手动输入很长的命令。这时她发现了神奇的构建工具 `Make`, 它通过一个名叫 `Makefile` 的文件中描述的规则来自动构建项目。当目标所需的依赖项发生改变时, `Make` 会自动帮我们通过给出的命令重新构建这个目标, 这样我们就不用手动一次次敲编译命令了。阮小妍编写了一个 `Makefile` 用于构建以上程序。

```
program : program.o tools.o
        gcc -o program program.o tools.o

program.o : program.c tools.h
        gcc -c program.c

tools.o : tools.c tools.h
        gcc -c tools.c

.PHONEY : clean

clean :
        -rm program main.o tools.o
```

(4) 你能说出 Makefile 规则的三个部分的作用吗 (3 分)

11. (10 分) 阮小妍和校科协的同学组成了一个团队，共同开发项目。在开发过程中，他们使用了 Git 作为版本控制工具，以 GitHub 作为远程仓库。

在这个仓库中，项目负责人已经建立了 main 分支和 dev 分支，并且开启了分支保护，只能通过 Pull Request 的方法向 dev 分支提交；此外还要求保持线性提交记录。阮小妍已经将仓库克隆到本地并切换到 dev 分支，请用命令行帮助阮小妍解决她遇到的问题。

(1) 现在阮小妍接到了分配给她的第一个任务，她需要一个任务对应的功能分支，名为 feature-branch，她该如何建立功能分支？(2 分)

(2) 因为这个任务有一定难度，阮小妍先完成了其中的一部分，并正确使用了 git add 指令，现在阮小妍应当如何提交她目前做出的所有更改到 GitHub 呢？(2 分)

(3) 在提交了所有更改后，阮小妍发现 dev 分支出现了新的提交，她应该如何合并这些提交到她的功能分支中呢？(2 分)

阮小妍终于完成了这个任务并提交了 Pull Request，她发现一共有三种选项可以用于合并 Pull Request: Create a merge commit, Squash and merge, Rebase and merge。

(4) 这些选项分别有什么作用呢，哪一个选项最适合她呢？(文字叙述) (2 分)

阮小妍完成了她的第一个任务，看着屏幕上显示的 Merged Pull Request，她非常高兴，并用以上的工作流程继续参与项目的开发。在经历了多次提交和测试后，整个项目终于完成了。但是阮小妍的心中还有一个疑惑：为什么在这么大的仓库里创建 Git 分支的速度还是和一开始一样快呢？

(5) 请你解答阮小妍的疑惑。(文字叙述) (2 分)

提高部分

1. (10 分)

Alice: I see the hacker you mean.

Bob: The new comer?

Alice: Yes. Take care. He/She has reached a higher level now. He/She can read our messages.

Bob: That doesn't matter. He/She thinks we are part of the Internet.

Alice: I like this hacker. He/She hacked well. He/She did not give up.

Bob: Now let's see. Try to solve the new challenge, hacker.

算出私钥，找到 Alice 和 Bob 想告诉你的信息。写出答案及相应代码

```
from flag import msg
from Crypto.Util.number import *

p=getPrime(64)
q=getPrime(64)
n=p*q
e=65537
m=bytes_to_long(msg)
c=pow(m,e,n)

print("n= ",n)
print("c= ",c)

"""
n=  138248481169135450772910654019458087329
c=  66674872827551010127258331506362966133
"""
```

- (1) 算出 p 和 q 的值。(4 分)
- (2) 算出 m 的值。(2 分)
- (3) 算出相应的明文。(2 分)
- (4) 给出第二、三小题相应的代码。(2 分)

2. (10 分) 八皇后问题是经典的算法问题，由国际象棋棋手马克斯·贝瑟尔于 1848 年提出：在 8×8 格的国际象棋上摆放 8 个皇后，使其不能互相攻击，即任意两个皇后都不能处于同一行、同一列或同一斜线上，问有多少种摆法。

- (1) 高斯认为有 76 种方案。你认为有多少种方案？给出答案并说明思路。(3 分)
- (2) 若用 n 替换 8，又有多少种方案呢？（其中 $n \geq 1$ ）请用编程实现求解，并说明所用算法、解释相应步骤思路。(7 分)

3. (10 分) 位于南一食堂的阮小妍等不及要去吃麦当了,但她还不是很熟悉学校的布局,需要你来为她指指路。

C++ 提供了一个有意思的语言特性:运算符重载。例如, C++ 标准库 `iostream` 为我们提供了很方便的流式 IO 操作, 让我们可以写出如下的代码:

```
int a, b;
std::cin >> a >> b;
```

聪明的你一定可以使用 `Person` 类和 `Location` 类, 通过重载 `>>` 运算符来在代码中生动形象地为阮小妍指路吧?

规则: 若阮小妍 `Rxy` 当前位置为 `A`, 那么 `Rxy >> B` 表示她试图从 `A` 走到 `B`。如果可以直达 `B`, 那么下次出发阮小妍起点为 `B`; 如果不能直达 `B`, 那么下次出发阮小妍起点还是 `A`。

请你根据给出的信息, 结合南邮仙林校区的布局, 阅读并完善以下代码, 并在主函数中写出阮小妍去麦当的正确走法 (走法不绝对, 合理就行, just for fun!) (10 分)

```
#include <iostream>
#include <string>

struct Location {
    std::string name;
    bool isConnected(const Location& other) const {
        // 具体实现此处省略, 用于判断两点是否可达
    }
};

struct Person {
    std::string name;
    const Location* cntLocation; // 人物当前所在地点
};

_____ operator>>(Person& p, const Location& target) {
    if (!p.cntLocation || _____) {
        _____;
        std::cout << p.name << "成功走到了" << target.name <<
            std::endl;
    } else {
        std::cout << p.name << "没法从" << p.cntLocation->name <<
            "走到" << target.name << std::endl;
    }
    return _____;
}
```

```

int main() {
    Person Rxy{"阮小妍", nullptr};
    Location KFC{"肯德基"};
    Location MC{"麦当劳"};
    Location Nan1{"南一食堂"};
    Location Nan2{"南二食堂"};
    Location Nan3{"南三食堂"};
    Location Library{"图书馆"};
    Location NanCao{"南操"};
    // 根据给出的地点以及南邮布局，给出一种你认为合理的走法吧！
    Rxy >> Nan1 >> _____;
}

```

4. (10 分) 动画器 (Animator Controller) 是 Unity 用来控制动画播放的好帮手，阮小妍制作了一款 2d 游戏，其主角有待机 (Idle)，行走 (Walking)，思考 (Thinking) 三种动画

(1) 请你和阮小妍合作，在下图中帮她添加动画状态，并由阮小妍设置所有的参数 (Parameters)，完成这个动画控制器 (3 分)



阮小妍觉得单纯的放动画有些无聊，想给主角加上音效，她打算给整个行走 (Walking) 状态加上走路的音效。

(2) 请你基于动画控制器帮她写出实现的办法 (只需给出思路，无需写出代码) (3 分)

阮小妍在添加动画状态时联想到了 FSM (有限状态机)，她决定尝试用 FSM 去设计一个失乡骑士 (类名: Knight) 的简单游戏 AI，包含四种状态: 巡逻 (Patrol)、追逐 (Chase)、攻击 (Attack)、死亡 (Die)

(3) 根据游戏常识，请按照下表格式为其设计状态变换表，或仿照第一小问绘制示意图，并尝试用代码实现你的设计。(以下空白一行仅供示例，请根据需求自行设计) (4 分)

当前状态	条件	状态转移

5. (10 分) 阮小妍在当地的一家动物园里工作, 动物们每隔一段时间需要喂食, 随着动物越来越多, 决定何时该投喂哪些动物成了阮小妍的一大烦恼, 于是她打算使用 C# 制作一个动物园管理系统, 帮助自己筛选出来需要投喂的动物。于是阮小妍设计了一个 Animal 类:

```
public class Animal
{
    public string Type {get; set;} // 是哪种动物
    public int Id {get; set;} // 动物的编号
    public int FeedingCycle {get; set;} //
        每隔多少小时需要喂食一次
    public int LastFeeding {get; set;} // 距上次投喂过了多少小时
}
```

她将所有动物的信息都装入了一个 `List<Animal> animalsList` 列表中, 得知可以用 `foreach` 来遍历列表中的每一个元素:

```
List<int> needFeeding = [];
foreach (var i in animalsList)
{
    if (i.FeedingCycle < i.LastFeeding)
        needFeeding.Add(i);
}
```

但是阮小妍所使用的编辑器居然在 `foreach` 处标注了波浪线, 建议 “Convert into LINQ-expression”, 即可以转换为 LINQ 查询表达式, 她应用了这个建议后, IDE 把这几行代码改成了简单的一个语句! 学习过 SQL 语句的阮小妍对这个语法感到无比熟悉, LINQ 正是集成于 C# 语言的一种类似于 SQL 的查询语法。(甚至可以直接转换为 SQL 语句执行!) 在略微修改和添加注释后, 代码变成了这样:

```
var needFeeding = from i in animalsList
                   where i.FeedingCycle < i.LastFeeding
                   select i.Id;
// 第一行: 从 animalsList 依次选择每一个元素, 命名为 i
// 第二行: 如果元素满足 i.FeedingCycle < i.LastFeeding,
//          则用来构建新 “列表”
// 第三行: 把构建的新 “列表” 中的每一个 Animal 的 Id 提取出来,
//          构建一个新 “列表”
```

但是 IDE 又给出了一个重构选项: “Convert LINQ to method chain”, 阮小妍查阅资料后得知 LINQ 除了上面这种非常类似 SQL 的查询语法以外, 还有一种方法语法, 可以在必要时完成更复杂的任务。

(1) 请你写出来这一 LINQ 表达式的方法语法形式。(3 分)

```
var needFeeding =
```

在使用 LINQ 的方法语法时,阮小妍发现所有实现了 `IEnumerable<T>` 接口的列表/集合(比如 `List<T>`)都可以使用 LINQ 的相关方法。阮小妍认为是 `IEnumerable<T>` 接口内定义了相关方法,可是在阅读了该接口的定义后却找不到任何 LINQ 相关方法的定义!删除代码最开头引入的 `System.Linq` 命名空间后,更蹊跷的事情发生了——在 `Where`、`Select` 方法上,IDE 提示 `IEnumerable<Animal>` 不包含该方法的定义,并且没有一个扩展方法接受 `IEnumerable<Animal>` 作为第一个参数。

前半部分很好理解,但是扩展方法是什么?查阅资料后得知通过扩展方法可以在不对相应类的代码进行修改的情况下,为一个类添加新的方法,并且阮小妍发现原来 LINQ 的相关方法都是扩展方法。

阮小妍发现在自己的管理系统中多处都要用到这个筛选功能,尽管它简短到只需要一行,但为了提高软件的可维护性,她决定把这个语句封装成一个 `IEnumerable<int>` 的扩展方法,命名为 `GetAnimalsToFeedIds`。这样,作为 `IEnumerable<int>` 的方法,它可以像 LINQ 的方法语法一样便捷的被使用,既符合直觉又优雅。

(2) 请帮她把 `GetAnimalsToFeedIds` 补充参数列表和函数体。(3 分)

```
public static class AnimalExtensions {
    // 请完成该方法的相应代码, 以及参数列表
    public static IEnumerable<int>
        GetAnimalsToFeedIds(_____)
}
// 调用样例:
// var needFeeding = animalsList.GetAnimalsToFeedIds();
```

阮小妍觉得只能筛选出动物编号并不方便,她希望还能同时筛选出来是哪种动物,即 `Animal` 的 `Type` 属性。为控制 `Animal` 类的职责以提高可维护性,并且避免将不必要的信息传递给需要使用这个方法的模块,你不能使用 `Animal` 类作为返回类型(可以设计新类型,不过这不是必要的)。

(3) 请帮助阮小妍构建 `needFeeding` 列表,列表中每一项应该同时包含 `Id` 和 `Type` 的值。(4 分)

```
// 在构建出来的needFeeding列表中, 应该同时包含Id和Type的值
// 你可以设计新的类型(class, struct, record等均可)
var needFeeding =
```

6. (10 分) MVP (Model、View、Projection) 是图形学中的基础变换, 将数据从模型空间变换到 NDC。阮小妍正在编写 C++ 程序, 使用 eigen 库来完成一个特定的 MVP 变换。请阅读以下代码, 帮助阮小妍完成 Model 和 Projection 变换, 其中 `Matrix4f` 表示数据类型是一个 4×4 大小, 元素为 `float` 类型的矩阵。

```
float DEG2RAD(float angle) {
    return (3.1415926*angle/180);
}

Matrix4f getModelMatrix(float rotationAngle){
    Matrix4f model = Matrix4f::Identity();
    //你需要在第2问完成该函数
    return model;
}

Matrix4f getProjectionMatrix(float eyeFov, float aspectRatio,
    float zNear, float zFar){
    Matrix4f projection = Matrix4f::Identity();
    //你需要在第3问完成该函数
    return projection;
}
```

(1) 阮小妍学习图形学知识时发现, 明明图形在三维空间中进行变换, 却使用了四维空间的向量和矩阵进行描述。这种技巧叫做齐次坐标, 你能说说齐次坐标的相较于三维坐标在进行图形变换时的优点吗? (2 分)

(2) 帮帮阮小妍完成函数 `getModelMatrix`, 这个函数接受一个角度 `rotationAngle` (deg), 输出一个矩阵, 该矩阵能使点在 `xOy` 平面旋转对应的角度。(只需要考虑 `x,y` 坐标轴的旋转) (你可以使用任意编程语言、伪代码、数学符号回答) (4 分)

(3) 帮帮阮小妍完成函数 `getProjectionMatrix`, 它代表了从观察空间到标准设备空间的变换, 同时我们规定相机投影于观察空间近平面的中心。(你可以使用任意编程语言, 伪代码, 数学符号回答) (4 分)

7. (10 分) 阮小妍正在使用 Java 参与开发一个用于科协活动管理的平台, 负责编写 `Event` 类和部分 `Rule` 类, 用于存放活动信息和检查报名的要求。她非常高兴, 并编写了一部分代码, 其中 `Event` 类维护了一个活动列表 `EVENTS`。

```
class Participant {
    private String name;
    private String department;

    public Participant(String name, String department) {
        this.name = name;
        this.department = department;
    }
}
```

```

        public String getDepartment() {
            return department;
        }
    }

class Event {
    private String name;
    private String department;
    private Integer rating;
    private List<Rule> rules = new ArrayList<>();
    private List<Participant> participants = new ArrayList<>();

    private static final List<Event> EVENTS = new ArrayList();

    public Event(String name, String department) {
        this.name = name;
        this.department = department;
    }

    public void addRule(Rule rule) {
        rules.add(rule);
    }

    public void registerParticipant(Participant participant) {
        for (Rule rule : rules) {
            if (!rule.check(this, participant)) {
                throw new IllegalArgumentException("Participant
                    does not meet the rules");
            }
        }
        participants.add(participant);
    }
}

interface Rule {
    // 验证参与者是否符合规则，如果符合规则则返回 true
    boolean check(Event event, Participant participant);
}

```

阮小妍在编写 Event 类的 `getEvents` 方法时觉得使用 for 循环的方法不够直观简洁，这时她想到了以前看过的 Streams API 和 Lambda 表达式。Streams API 提供了一种函数式编程风格，可以用链式的方式处理集合，使代码简洁、易读；同时 Lambda 表达式可以方便地实现函数式编程，和 Streams API 一起使用的时候可以发挥强大的力量。于是阮小妍

使用 Streams API 为 Event 类添加了 `getEvents` 方法，用于获取特定部门的活动列表。

```
public static List<Event> getEvents(String department) {  
    return EVENTS.stream()  
        .filter(event ->  
            event.department.equals(department))  
        .collect(Collectors.toList());  
}
```

(1) 现在请你学习 Streams API 的用法，并跟随阮小妍的代码，使用 Streams API 为 Event 类添加 `getHighestRatingEvent` 方法，用于从活动列表中筛选出某个部门评分最高的活动。(2 分)

```
public static Event getHighestRatingEvent(String department){  
  
}
```

阮小妍在编写相应的 Rule 类时，发现 Event 类需要处理不同的预约规则，于是她做出了这样的设计：每个活动（Event）可以关联多种不同的预约规则（Rule），例如最大参与者数量、限制参与者来自特定部门等，虽然规则不同，但都实现共同的接口 Rule，以保证活动预约系统的灵活性和可扩展性。按照这个设计，阮小妍检查了目前需要的规则，并添加了不完整的 `MaxParticipantRule` 类用于限制参与者数量，以及不完整的 `DepartmentRule` 类用于保证只有特定部门的参与者可以参加活动。

(2) 请你根据以上原则完善 `MaxParticipantRule` 类和 `DepartmentRule` 类，并让这两个类实现 Rule 接口。(8 分)

```
class MaxParticipantRule _____ {  
    private int maxParticipants;  
  
    public MaxParticipantRule(int maxParticipants) {  
        this.maxParticipants = maxParticipants;  
    }  
    // 在这里补充代码  
}  
  
class DepartmentRule _____ {  
    private String requiredDepartment;  
  
    public DepartmentRule(String requiredDepartment) {  
        this.requiredDepartment = requiredDepartment;  
    }  
    // 在这里补充代码  
}
```

8. (10 分) 阮小妍正在开发一个数据分析平台，需要从服务器获取数据、处理数据，并将结果展示给用户。现在她已经写好了部分核心代码，包含三个函数，它们的名称和功能分别为：

- `fetchData`：模拟异步获取某部门活动数据的过程。
- `processData`：模拟数据值的处理。
- `displayResult`：模拟展示处理结果。

相应代码如下。

```
// 模拟获取数据的函数
function fetchData(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      const data = { id, value: Math.random() * 100 };
      resolve(data);
    }, 1000);
  });
}

// 模拟处理数据的函数
function processData(data) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (data.value > 50) {
        resolve(`Processed data: ${data.value}`);
      } else {
        reject(`Data value too low: ${data.value}`);
      }
    }, 1000);
  });
}

// 模拟显示结果的函数
function displayResult(result) {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log(result);
      resolve();
    }, 500);
  });
}
```

阮小妍已经把重要的函数写好了，你只要在这里调用函数按序依次获取数据、处理数据和显示结果；如果抛出错误，需要捕获后输出。

(1) 这是阮小妍学姐写了一半的代码，请帮她补充完整。(3 分)

```
fetchData(1)
  .then(processData)
  // 在这里添加代码
```

JS 中的 `async` 和 `await` 关键字可以用一种更简洁的方式写出基于 `promise` 的异步行为，而无需链式调用 `promise`。请你使用这两个关键字将刚才的代码封装在一个函数中，并在原处通过调用函数。

(2) 尝试用 `async` 和 `await` 重写第一小题包含的代码。(3 分)

阮小妍还需要完成另一个功能，需要同时处理多个部门的数据（例如部门编号为 1、2、3），在所有数据处理完成后再输出结果，并在所有任务完成后输出 “All tasks completed”。

(3) 请完成 `executeAll` 函数实现这一功能。(4 分)

```
async function executeAll(departmentIds) {
  // 在这里添加代码
}

executeAll([1, 2, 3]);
```

9. (10 分) `awk`, `sed` 和 `grep` 通常被称为 Linux 文本三剑客，通过熟练的运用这三个工具，可以便利地完成许多复杂的文本操作。

阮小妍学姐忽然收到了组里平日十分腼腆的男孩的一份英文情书 `letter.txt`，你能帮帮学姐利用 `shell` 命令回复他吗？（要求命令在一行内完成，不允许执行外部脚本文件）

(1) 利用 `shell` 工具，统计情书中 `ruanxiaoyan`, `Xiaoyan`, `love u` 的总个数，并将统计结果输出到当前目录中的 `my_909_loveletter.txt` 中。(5 分)

(2) 利用 `shell` 工具，在情书中每一行 `would you love me?` 后的下一行给出你的回复，并将情书以 `TCP` 方式回复给主机名为 `chandler`，端口为 `34031` 的主机（回复内容不限）(5 分)

10. (10 分) 阮小妍学姐正在开发一个应用，但她遇到了几个问题，分别需要用到以下几个特性：装饰器、生成器和上下文管理器。你能帮她解决吗？

(1) 编写一个装饰器 `timer_decorator`，用来计算一个函数的执行时间。装饰器应接受一个函数作为参数，并返回一个新的函数，该函数在执行时会打印出执行时间。(3 分)

```
import time

def timer_decorator(func):
    # 请在这里完成代码
    pass

@timer_decorator
def example_function(n):
    time.sleep(n)
    return f"Slept for {n} seconds"

# 测试代码
example_function(2)
```

(2) 编写一个生成器函数 `fibonacci_generator`，用于生成斐波那契数列中的前 `n` 项。生成器应逐个生成斐波那契数，而不是一次性返回整个序列。(3 分)

```
def fibonacci_generator(n):
    # 请在这里完成代码
    pass

# 测试代码
for number in fibonacci_generator(5):
    print(number)
```

(3) 实现一个上下文管理器 `file_reader`, 用于安全地打开和关闭一个文件。上下文管理器应当支持 `enter` 和 `exit` 方法, 以便在使用 `with` 语句时自动管理文件的打开和关闭。(4 分)

```
class file_reader:
    def __init__(self, filename):
        self.filename = filename

    def __enter__(self):
        # 请在这里完成代码
        pass

    def __exit__(self, exc_type, exc_value, traceback):
        # 请在这里完成代码
        pass

# 测试代码
with file_reader('example.txt') as f:
    content = f.read()
    print(content)
```